LEMBAR KERJA PRAKTIKUM 6 STRUKTUR DATA

SORTING DALAM STRUKTUR DATA

A. Tujuan Praktikum

Setelah mengikuti praktikum ini, mahasiswa mampu:

1. Mengimplementasikan algoritma sorting dalam bahasa C.
2. Menganalisis perbedaan implementasi sorting pada array dan linked list.
3. Memahami pengaruh struktur data terhadap kompleksitas waktu dan ruang.
4. Mengamati konsep stabilitas dalam pengurutan data record.

B. Implementasikan Program di bawah ini

Program 1. Insertion Sort dan Selection Sort pada Array

```
#include <iostream>
#include <vector>
using namespace std;

long long compIS = 0, shiftIS = 0;
long long compSS = 0, swapSS = 0;

void printArray(const vector<int>& arr) {
    for (int x : arr)
        cout << x << " ";
    cout << endl;
}

void insertionSort(vector<int>& arr) {
    int n = arr.size();

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0) {
            compIS++;
            if (arr[j] > key) {
                arr[j + 1] = arr[j];
                shiftIS++;
                j--;
            } else {
                break;
```

```
            }
        }
        arr[j + 1] = key;
    }
}

void selectionSort(vector<int>& arr) {
    int n = arr.size();

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        for (int j = i + 1; j < n; j++) {
            compSS++;
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }

        if (minIndex != i) {
            swap(arr[i], arr[minIndex]);
            swapSS++;
        }
    }
}

int main() {
    int n;
    cout << "Masukkan jumlah data: ";
    cin >> n;

    vector<int> arr1(n), arr2(n);

    cout << "Masukkan data:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr1[i];
        arr2[i] = arr1[i];  // copy untuk selection sort
    }

    insertionSort(arr1);
    selectionSort(arr2);

    cout << "\nHasil Insertion Sort:\n";
    printArray(arr1);
    cout << "Perbandingan: " << compIS << endl;
    cout << "Pergeseran: " << shiftIS << endl;
```

```
    cout << "\nHasil Selection Sort:\n";
    printArray(arr2);
    cout << "Perbandingan: " << compSS << endl;
    cout << "Swap: " << swapSS << endl;

    return 0;
}
```

Setelah Anda mengimplementasikan program di atas, lakukan pengamatan terhadap perbedaan jumlah operasi dengan menggunakan tiga jenis data uji:

- Data A: Data acak
- Data B: Data sebagian besar terurut
- Data C: Data sudah terurut naik
- Data D: Data terurut turun

Program 2. Insertion Sort pada Linked List

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

long long compLL = 0;

Node* createNode(int data) {
    Node* newNode = new Node;
    newNode->data = data;
    newNode->next = nullptr;
    return newNode;
}

Node* insertLast(Node* head, int data) {
    Node* newNode = createNode(data);

    if (head == nullptr)
        return newNode;

    Node* temp = head;
    while (temp->next != nullptr)
        temp = temp->next;
```

```
    temp->next = newNode;
    return head;
}

void printList(Node* head) {
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}

Node* insertionSortList(Node* head) {
    Node* sorted = nullptr;

    while (head != nullptr) {
        Node* current = head;
        head = head->next;

        if (sorted == nullptr || sorted->data >= current->data) {
            compLL++;
            current->next = sorted;
            sorted = current;
        } else {
            Node* temp = sorted;
            while (temp->next != nullptr && temp->next->data <
current->data) {
                compLL++;
                temp = temp->next;
            }
            current->next = temp->next;
            temp->next = current;
        }
    }

    return sorted;
}

int main() {
    Node* head = nullptr;
    int n, x;

    cout << "Masukkan jumlah data: ";
```

```
    cin >> n;

    cout << "Masukkan data:\n";
    for (int i = 0; i < n; i++) {
        cin >> x;
        head = insertLast(head, x);
    }

    cout << "Sebelum sorting:\n";
    printList(head);

    head = insertionSortList(head);

    cout << "Setelah sorting:\n";
    printList(head);

    cout << "Jumlah perbandingan: " << compLL << endl;

    return 0;
}
```

Lakukan pengamatan yang sama dengan Program 1 sebelumnya. Gunakan variasi data yang sama pada Program 1 untuk dijalankan pada Program 2.

C. Berdasarkan program yang telah Anda jalankan, lakukan proses tracing dan gunakan fitur watch variables untuk mengamati alur kerja program secara detail. Setelah memahami prosesnya, diskusikan pertanyaan-pertanyaan berikut bersama asisten dan teman-teman Anda.

1. Analisis Implementasi
   a. Apakah implementasi Insertion Sort pada linked list lebih mudah atau lebih sulit dibandingkan pada array? Jelaskan alasannya.
   b. Operasi apa yang paling dominan pada Insertion Sort versi array?
   c. Operasi apa yang paling dominan pada Insertion Sort versi linked list?
   d. Mengapa pada linked list tidak terdapat proses shifting seperti pada array?

2. Analisis Perilaku Algoritma
   a. Mengapa Insertion Sort bersifat adaptif terhadap data yang hampir terurut?
   b. Mengapa proses shifting pada array dianggap mahal, sedangkan pada linked list tidak terjadi shifting?
   c. Bandingkan jumlah operasi (perbandingan, pergeseran, atau manipulasi pointer) pada kedua struktur data.

3. Analisis Kompleksitas dan Performa
   a. Apakah kompleksitas teoretis selalu mencerminkan performa nyata program? Jelaskan berdasarkan hasil percobaan Anda.
   b. Bagaimana pengaruh struktur data terhadap jumlah operasi yang terjadi?
   c. Berdasarkan pengamatan Anda, struktur data mana yang lebih sesuai untuk Insertion Sort? Mengapa?

D. Video dan Hasil Diskusi (dikumpulkan selambatnya satu minggu setelah praktikum ini)

Setelah Anda mempelajari program di atas, kerjakan tugas di bawah ini dengan kelompok Anda, lalu buat video diskusinya dan tuliskan hasil diskusinya (PDF), masukkan ke gdrive, lalu kumpulkan link gdrive pada form berikut: https://forms.gle/JCWa8XJy9n7Vnc7r7

Tugas. Modifikasi Program Menggunakan Record

Ubah Program 1 (array) sehingga data yang disorting bukan lagi bertipe int, tetapi berbentuk record berikut:

```
struct Mahasiswa {
    string nama;
    int nilai;
};
```

Sorting dilakukan berdasarkan nilai (ascending).

Gunakan data uji berikut:

Nama	Nilai
Andi	80
Budi	75
Citra	80
Deni	70
Eka	75

Observasi Hasil Sorting

1. Setelah menggunakan **Insertion Sort**, apakah urutan mahasiswa yang memiliki nilai sama tetap mempertahankan urutan awalnya?
2. Setelah menggunakan **Selection Sort**, apakah urutan mahasiswa dengan nilai yang sama tetap sama seperti sebelum sorting?
3. Mengapa stabilitas penting ketika data berbentuk record?
4. Mengapa Insertion Sort secara alami bersifat stabil?
   Mengapa Selection Sort pada implementasi standar tidak stabil?
   Modifikasi seperti apa yang dapat membuat Selection Sort menjadi stabil?
5. Lakukan sorting dua kali, urutkan berdasarkan **nama k**emudian urutkan berdasarkan **nilai.** Apakah urutan nama tetap konsisten untuk nilai yang sama? Algoritma mana yang mempertahankan urutan tersebut?
6. Mengapa stabilitas tidak terlihat ketika hanya menggunakan integer?
7. Simpulkan hubungan antara:
   - Struktur data
   - Cara pemindahan elemen
   - Stabilitas algoritma